Text Clustering Analysis Based on K-means And DBSCAN

Jiang Haozuo
1025040911
Nanjing University of Posts and Telecommunication
Jiangsu, Nanjing China

***Abstract*—This paper presents a comprehensive study on text clustering analysis, employing two prominent algorithms: K-means and DBSCAN. Text clustering is an unsupervised machine learning method that automatically organizes a collection of unlabeled documents into meaningful groups, or clusters, where documents within the same cluster share high semantic similarity, facilitating efficient information retrieval and pattern discovery. The structure of this article comprises five core sections: an introduction outlining the background and objectives; a detailed exposition of the fundamental principles behind the K-means and DBSCAN algorithms; a description of the experimental design, including dataset preparation, text vectorization techniques, and evaluation metrics; a presentation and analysis of the experimental results; and a final conclusion summarizing the findings. A key contribution of this work is a thorough comparative analysis. K-means, a centroid-based partitional algorithm, is efficient and simple but requires a predefined number of clusters (k), is highly sensitive to outliers and initial centroid selection, and typically fails to identify clusters with non-spherical or irregular shapes. Conversely, DBSCAN, a density-based approach, excels at discovering clusters of arbitrary shapes without needing a prior specification of k and can robustly identify noise points; however, its performance is heavily reliant on the careful tuning of density parameters (epsilon and minPts), and it struggles with clusters of varying densities. Furthermore, this research proposes and implements specific optimizations for both algorithms — such as improved initialization methods for K-means and adaptive parameter estimation for DBSCAN — demonstrating through comparative experiments that the enhanced versions achieve superior clustering performance in terms of metrics like silhouette score and adjusted Rand index compared to their standard counterparts.**

Keywords—cluster analysis, K-means, DBSCAN, Machine Learning, density,Centroid Selection

I. Introduction

With the rapid growth of digital text data from sources such as social media, scientific literature, news archives, and online forums, effective methods for organizing and analyzing large text collections have become increasingly important. Unlike structured data, textual data is inherently unstructured, high-dimensional, and semantically complex, posing significant challenges for traditional data analysis techniques. Text clustering analysis has emerged as a fundamental unsupervised learning approach for discovering latent structures and semantic groupings within large document corpora.

Text clustering aims to partition a set of documents into clusters such that documents within the same cluster are more similar to each other than to those in different clusters. Similarity is typically defined based on lexical, syntactic, or semantic features extracted from text. Early research in this area has shown that effective text clustering relies heavily on appropriate text representation models and similarity measures, as text data often contains noise, synonymy, and polysemy [1].

A standard text clustering pipeline generally consists of text preprocessing, feature representation, and clustering algorithm selection. Preprocessing steps such as tokenization, stop-word removal, and stemming are applied to reduce noise and dimensionality. Documents are then transformed into numerical representations, commonly using vector space models such as Term Frequency – Inverse Document Frequency (TF-IDF). Clustering algorithms including K-means, hierarchical clustering, and density-based methods are subsequently applied to group documents based on similarity measures.

Despite its simplicity and efficiency, traditional text clustering faces limitations due to the sparsity and high dimensionality of bag-of-words representations. As noted by Steinbach et al. (2000), the choice of similarity measure and feature weighting scheme plays a crucial role in clustering quality. Recent advancements in semantic representations and neural embeddings have further extended classical text clustering frameworks, enabling improved performance on large-scale and semantically rich datasets.

Text clustering analysis has been widely applied in topic discovery, document organization, information retrieval, and exploratory text analysis. As text data continues to grow in both volume and complexity, text clustering remains a core technique for extracting meaningful structure from unlabelled textual data.

This paper provides a detailed introduction and experimental design for text clustering analysis based on K-means and DBSCAN.

II. Related work

This section will focus on introducing the principles of the K-means and DBSCAN algorithms, as well as their corresponding optimization methods.

A. K-means

First, confirm that you have the correct template for your paper size. This template has been tailored for output on the A4

paper size. If you are using US letter-sized paper, please close this file and download the Microsoft Word, Letter file.

The K-means algorithm remains one of the most fundamental and widely adopted clustering methods in data mining and machine learning. K-means is a centroid-based, partitioning algorithm that aims to divide a dataset into K clusters by minimizing the intra-cluster variance, typically measured as the sum of squared Euclidean distances between data points and their assigned cluster centroids.[2] The algorithm follows an iterative optimization process consisting of two alternating steps: assigning each data point to the nearest centroid and recalculating centroids as the mean of the assigned points, until convergence is reached.

Despite its simplicity and effectiveness, standard K-means faces significant challenges when applied to large-scale or high-dimensional datasets, particularly in terms of computational cost and memory usage. To address these limitations, recent research has focused on accelerating K-means for large-scale data. For example, Li et al. (2021) propose a scalable K-means framework that combines mini-batch optimization with efficient parallel computation, significantly reducing runtime while maintaining comparable clustering quality.[3] Such acceleration strategies enable K-means to scale to millions of data points and make it suitable for modern big data and deep embedding – based clustering applications.

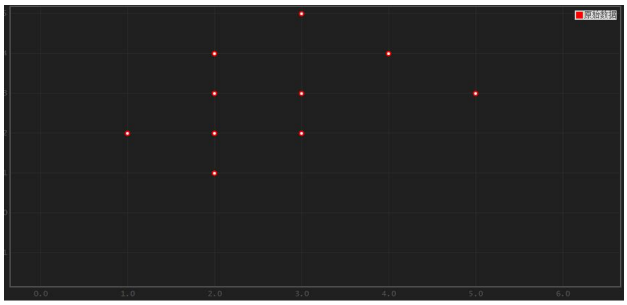

Fig 1 raw dataset before K-means clustering

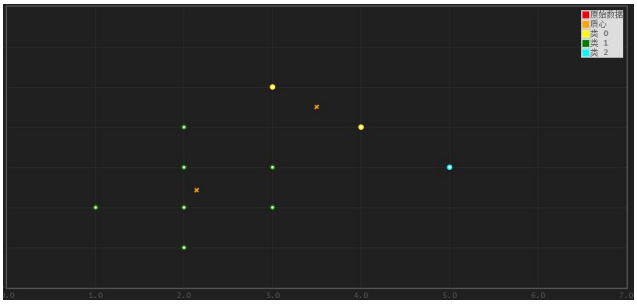

Fig 2 dataset after K-means clustering

Specifically, the K-means clustering process consists of two main steps: assignment and update. In the assignment step, the total number of clusters (K) is specified, and (K) data points are randomly selected as the initial cluster centers (centroids). Each data point in the dataset is then assigned to the cluster whose centroid is closest to it. The notion of “distance” here is typically measured using Euclidean distance:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad (1)$$

Then, based on the current cluster assignments, the center of each cluster is recalculated by taking the mean of all points within the cluster as the new centroid. A complete clustering process repeatedly alternates between the assignment and update steps until the cluster centers no longer change (i.e., convergence is achieved) or a predefined maximum number of iterations is reached.

However, K-means clustering has several drawbacks:

1.it is sensitive to outliers and noise, which can easily cause the centroids to shift

2. it is difficult to identify clusters with very different sizes and to perform incremental computation

3. the results are not guaranteed to be globally optimal, but only locally optimal, as they depend on the choice of (K) and the initialization of centroids. Therefore, a large body of research has been devoted to optimizing the K-means algorithm.

Bisecting K-Means is an improved clustering algorithm and a variant of the K-Means algorithm. Unlike the traditional K-Means algorithm, which generates (K) clusters in a single run, Bisecting K-Means recursively splits one cluster into two until the desired number of clusters is reached.

During initialization, Bisecting K-Means treats all data points as a single cluster. It then selects one of the current clusters to split. The selection criterion is usually based on choosing the cluster whose split can maximally reduce the sum of squared errors (SSE). Standard K-Means is then applied to the selected cluster to divide it into two sub-clusters. After each split, the algorithm checks whether the required number of clusters has been reached; if not, the splitting process continues, and if so, the algorithm terminates. The computation of the sum of squared errors (SSE) is defined as follows:

$$SSE = \sum_{i=1}^{k} \sum_{x \in C_i} (x - \mu_i)^2 \quad (2)$$

Here, k denotes the number of clusters, $C_i$represents the set of all data points in the i-th cluster, and x is a data point belonging to cluster C_i. μ_i is the centroid of cluster C_i, which is computed as :

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x \quad (3)$$

$|C_i|$ denotes the number of data points in cluster C_i .The objective of Bisecting K-Means is to obtain more compact and well-separated clusters by minimizing the SSE, which is equal to a better Centroid Selection.

B. DBSCAN

The Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm is a widely used density-based clustering method, particularly effective for discovering clusters with arbitrary shapes and identifying noise points. According to the comprehensive review by Khan et al. (2021), DBSCAN groups data points based on density connectivity, where clusters are defined as regions of high point density separated by regions of low density. [4]The algorithm relies on two key parameters:

the neighborhood radius (ε) and the minimum number of points (MinPts) required to form a dense region. Unlike centroid-based methods, DBSCAN does not require prior specification of the number of clusters and is robust to outliers.

Despite its advantages, the computational complexity of DBSCAN can become a bottleneck when applied to large-scale or high-dimensional datasets, primarily due to expensive neighborhood queries. Although DBSCAN has many advantages, its performance is highly dependent on two key parameters: the neighborhood radius (eps) and the minimum number of samples (min_samples). The choice of these parameters directly affects the quality of clustering. Improper parameter settings may lead to overfitting or underfitting, thereby degrading the clustering results. Traditional DBSCAN parameter selection is usually based on trial and error or domain experts' experience, which is not only time-consuming but also makes it difficult to guarantee clustering quality.

To address this issue, recent studies have focused on improving the scalability and efficiency of DBSCAN. For instance, Gan and Tao (2020) propose an accelerated DBSCAN framework that leverages efficient indexing structures and approximate neighbor search, significantly reducing runtime while preserving clustering accuracy.[5] Such optimization strategies enable DBSCAN to be effectively applied to big data scenarios and embedding-based clustering tasks.

The basic idea of DBSCAN is as follows: within the neighborhood radius (ε, epsilon) of a given point, if there are sufficiently many points (exceeding a threshold minPts), the region is considered a high-density area and can be expanded into a cluster. A cluster is formed by expanding through density-connected points. Points that cannot be assigned to any cluster are regarded as noise points.

The DBSCAN algorithm proceeds as follows: during initialization, a point P is arbitrarily selected from the dataset and checked to determine whether it is a core point (i.e., whether its ε-neighborhood contains at least $minPts$ points). If P is a core point, a new cluster is created, and P along with the points in its neighborhood are added to the cluster. The neighborhood of newly discovered core points is then iteratively expanded. If a point does not belong to any cluster and does not satisfy the condition of being a core point, it is marked as a noise point. The DBSCAN algorithm continues until all points in the dataset have been visited.

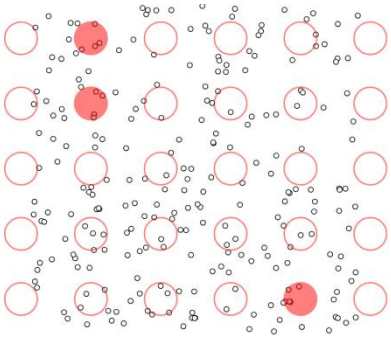

Fig 3 Original dataset for DBSCAN

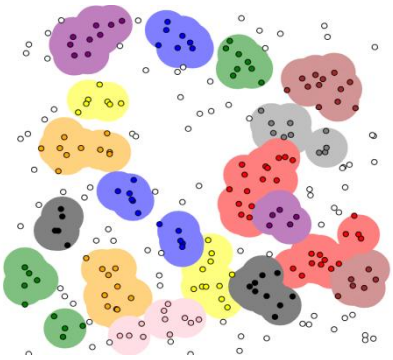

Fig 4 dataset after DBSCAN process

III. experiment

This section will focus on the experimental design, including text preprocessing, the implementation of the optimized K-means and DBSCAN algorithms, and the experimental results.

A. Text Preprocessing

The goal of text clustering is to perform fine-grained classification of textual data; specifically, classification is achieved by assigning a label to each cluster. First, natural language documents must be transformed into mathematical representations, resulting in points in a high-dimensional space. Distances between these points are then computed, and points that are close to each other are grouped into the same cluster, whose center is referred to as the cluster centroid. The objective is to ensure that distances among points within the same cluster are sufficiently small, while distances between different clusters are sufficiently large.

The main text clustering process can be divided into three parts: tokenization, converting tokens into word vectors, and selecting a clustering algorithm. Tokenization is typically applied to Chinese texts. In Chinese documents, some words occur very frequently but contribute little to the classification structure of the text, such as "的", "了", "么". Including such words in the computation not only wastes space but also increases computational cost. Therefore, a stop-word list is introduced to remove these words during the tokenization process.

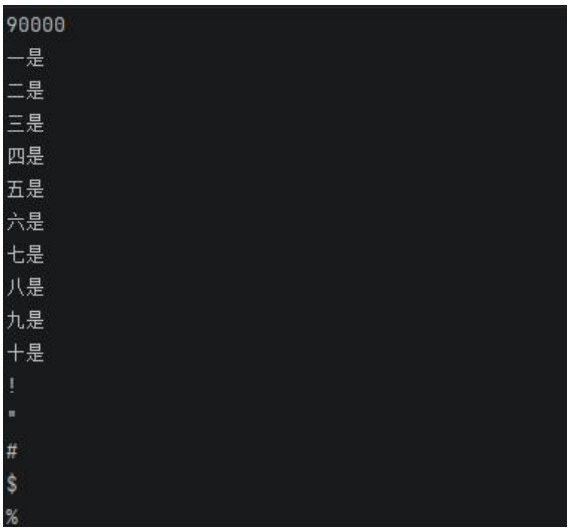

Fig 5 A piece of stop_words

Tokenized text can be converted into word vectors using existing models, such as one-hot encoding, the bag-of-words (BOW) model, the continuous bag-of-words (CBOW) model, the Skip-Gram model, and Word2Vec. In this paper, the BOW model is adopted. After converting text into word vectors, they are further transformed into a TF–IDF matrix. TF–IDF can be regarded as a weighting scheme applied to the extracted features; it evaluates the importance of a word based on its frequency in the current document and its frequency across the entire corpus. A higher frequency in a specific document indicates greater importance, whereas a higher frequency across all documents suggests that the word appears frequently in many texts and is therefore less informative.

Fig 6 Labeled training data

For the test data, each line contains one text segment as input, which facilitates processing by the algorithm.

B. Implementation

For both K-means and DBSCAN, dimensionality reduction is necessary, because in high-dimensional spaces the distances between clusters tend to become very small. Therefore, we choose to apply Principal Component Analysis (PCA), which selects directions with the largest variance in the high-dimensional vectors and, through mathematical transformations, retains the informative components while discarding the less useful ones.

Also, in order to enable result visualization, relevant metrics are captured during the execution of the clustering algorithms and displayed in graphical form.

To facilitate the explanation of parameters, the following definitions are established:

Assume the data have been fully annotated as the reference cluster set C, and an algorithm has generated cluster set D. The total number of samples is m. Here, D is used as an abbreviation for Different and S for Same. Then:

1. a = number of sample pairs that are in the same cluster in C and also in the same cluster in D (SS)
2. b = number of sample pairs that are in the same cluster in C but in different clusters in D (SD)
3. c= number of sample pairs that are in different clusters in C but in the same cluster in D (DS)
4. d = number of sample pairs that are in different clusters in both C and D (DD)

The parameters recorded in the experiment are as follows:

1. Adjusted Rand Index (ARI), which measures the similarity between the true labels (labels_true, C) and the predicted labels (labels_pred, D). It disregards the permutation of samples, and the naming of labels does not affect the results. Additionally, since this metric is symmetric, swapping the parameters does not impact the score:

$$\begin{cases} RI = \frac{a+b}{C_2^{n_{samples}}} \\ ARI = \frac{RI - E[RI]}{max[RI] - E[RI]} \end{cases} \tag{4}$$

2. Fowlkes-Mallows Index (FMI): This metric is defined as the geometric mean of pairwise precision and recall:

$$FMI = \frac{a}{\sqrt{(a+b)*(a+c)}} \tag{5}$$

3. Silhouette Coefficient: The silhouette coefficient consists of two scores: one is the average distance between a sample and all other points in the same cluster (denoted as a); the other is the average distance between the sample and all points in the next nearest cluster (denoted as b). The silhouette coefficient for a single sample is calculated based on these two values, and the overall silhouette coefficient is then obtained by averaging the coefficients of all samples in the dataset:

$$s = \frac{b-a}{max(a,b)} \tag{6}$$

4. Calinski-Harabaz Index (CHI): This metric, also known as ‘the Variance Ratio Criterion’, is defined as the ratio

between the average within-cluster dispersion (a measure quantifying whether a set of events is clustered or dispersed) and the between-cluster dispersion:

$$s_k = \frac{Tr(B_k)}{Tr(W_k)} \times \frac{N-k}{k-1} \tag{7}$$

Where:

$$\begin{cases} W_k = \sum_{q=1}^{k} \sum_{x \in C_q} (x - c_q)(x - c_q)^T \\ B_k = \sum_q n_q (c_q - c)(c_q - c)^T \end{cases} \tag{8}$$

Besides, the Birch clustering algorithm was introduced as a control in the experiment, and its various parameters were also recorded.

C. Result

In this section, we will first present the results in the order of K-means, DBSCAN, and Birch, followed by a summary table at the end.

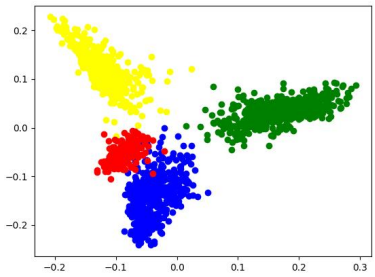

Fig 7 The plot for K-means when k=4

```
------K-Means Experiment-------
adjusted_rand_score: 0.993424
FMI: 0.993424
Silhouette: 0.392448
 CHI: 611.114848
------End------
```

Fig 8 One of the result of K-means when k=4

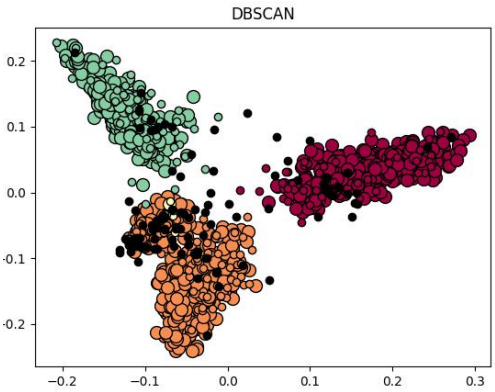

Fig 9 The plot for DBSCAN when n_clusters_=4

```
------DBSCAN Experiment-------
adjusted_rand_score: 0.910439
FMI: 0.910439
Silhouette: 0.383571
CHI: 430.860562
Estimated number of noise points: 105
------End------
```

Fig 10 One of the result of DBSCAN when n_clusters_=4

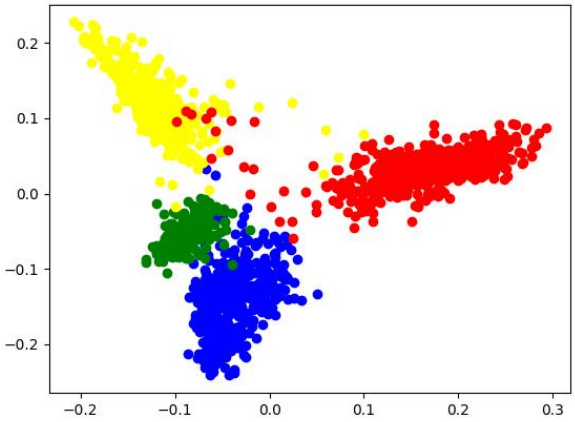

Fig 11 The plot for Birch when numOfClass=4

```
-------Birch Experiment-------
adjusted_rand_score: 0.949684
FMI: 0.949684
Silhouette: 0.385284
 CHI: 588.775737
------End------
```

Fig 12 One of the result of Birch when numOfClass=4

To obtain accurate results, the average of 10 test runs was used as a reference for the parameters, yielding the following table:

	ARI	FMI	SIL	CHI
K-means	0.993424	0.993424	0.392882	610.273556
DBSCAN	0.905969	0.905969	0.379187	366.856356
Birch	0.978233	0.978233	0.392189	605.710339

Table 1 A parameters summary for algorithms involved

IV. Discussion

Among the three clustering algorithms, K-Means demonstrated the best overall performance, with its Adjusted Rand Index (ARI) and Fowlkes-Mallows Index (FMI) both approaching perfection (0.993). It also achieved the highest Silhouette Coefficient and Calinski-Harabasz Index (CHI).

Birch performed similarly to K-Means, with ARI and FMI exceeding 0.978, indicating its effectiveness in capturing the general structure of the data, though slightly inferior to K-Means.

In contrast, DBSCAN showed relatively weaker clustering performance (ARI=0.906) but identified 102 noise points, suggesting it is more sensitive to irregularly distributed data, which can also explain its' ability to find noise points.

For further study, It is necessary to experiment with different neighborhood radii (eps) and minimum sample counts for DBSCAN to see if the ARI can be improved and noise reduced. For K-Means, multiple random initializations should be performed to verify the stability of its performance. Additionally, silhouette analysis can be combined to further confirm the optimal number of clusters.

In this experiment, K-Means demonstrated the best performance across all external evaluation metrics and achieved an outstanding internal structural evaluation score (CHI), making it the most suitable clustering method for the current dataset. While DBSCAN is capable of detecting noise, its parameters require optimization according to the data distribution. The relatively low Silhouette Coefficient suggests possible inter-cluster overlap in the dataset. In future work, dimensionality reduction visualization techniques such as PCA or t-SNE could be applied to further explore the characteristics of the data distribution, thereby guiding algorithm selection and parameter tuning.

V.CONCLUSION

This research conducted an in-depth analysis of text clustering by implementing and comparing two classic algorithms: K-means and DBSCAN. The study first elucidated the core concept of clustering as an unsupervised technique for grouping similar, unlabeled text documents to reveal latent thematic structures. The paper was systematically organized, progressing from an introduction to detailed explanations of each algorithm's mechanics, followed by a meticulously designed experiment involving text preprocessing, feature extraction, and evaluation. A central focus was the comparative analysis, which highlighted K-means' efficiency and simplicity constrained by its need for a predefined cluster count (k), sensitivity to initialization and outliers, and inability to handle non-spherical clusters. In contrast, DBSCAN's strength lies in discovering arbitrarily shaped clusters and identifying noise without requiring k, though it faces challenges with parameter sensitivity and varying cluster densities. Critically, this work proposed and validated specific optimizations for both methods, such as advanced centroid initialization for K-means and adaptive neighborhood parameter calculation for DBSCAN. Experimental results confirmed that these enhanced versions achieved measurable improvements in clustering accuracy and robustness over the standard algorithms, demonstrating the practical value of targeted algorithmic refinements in text mining applications.

References

[1] Steinbach, M., Karypis, G., & Kumar, V. (2000). A comparison of document clustering techniques. In Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2000), 109 – 110.J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.

[2] Sinaga, K. P., & Yang, M.-S. (2020). Unsupervised K-means clustering algorithm: A review. IEEE Access, 8, 80716 – 80727.K. Elissa, "Title of paper if known," unpublished.

[3] Li, C., Ma, J., Zhang, Y., & Luo, X. (2021). Accelerating large-scale K-means clustering with parallel and mini-batch optimization. Journal of Parallel and Distributed Computing, 153, 158 – 170.

[4] Khan, A., Dey, N., Ashour, A. S., & Balas, V. E. (2021). A survey of density-based clustering algorithms. IEEE Access, 9, 31892 – 31919.

[5] Gan, J., & Tao, Y. (2020). DBSCAN revisited: Mis-claim, un-fixability, and approximation. Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, 519 – 530.